

ARQUITETURAS DE COMPUTADORES

TURMA A1

TÓPICO 3 – MEMÓRIA CACHE (AULA 17)

Desempenho arquitetural

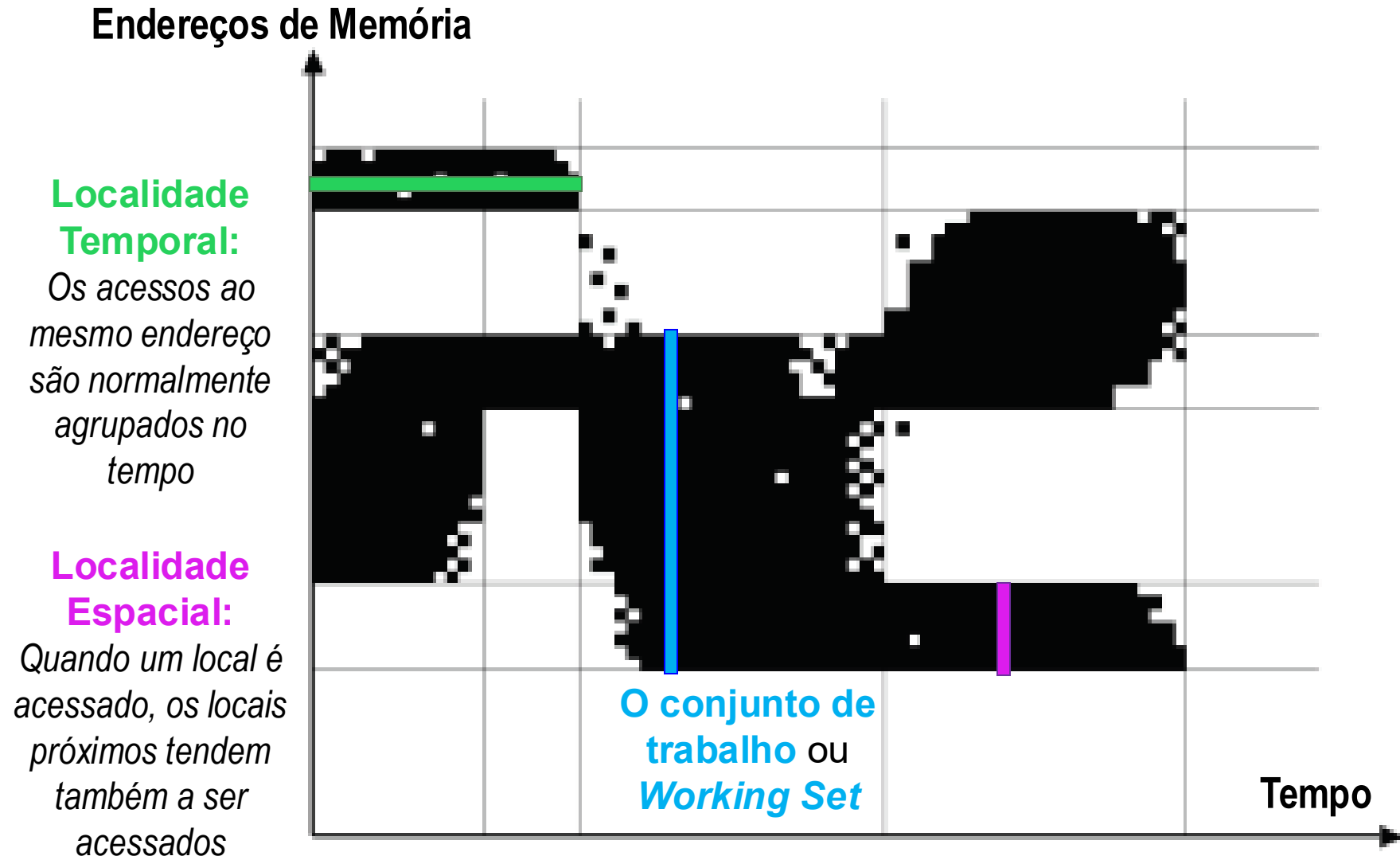
- O tempo de execução depende nas características do sistema todo, mas onde podemos achar melhorias?

$$\frac{\text{Tempo}}{\text{Programa}} = \boxed{\frac{\text{Instruções}}{\text{Programa}}} \times \boxed{\frac{\text{Ciclos}}{\text{Instrução}}} \times \boxed{\frac{\text{Tempo}}{\text{Ciclo}}}$$

Depende no: *Compilador* *Microarquitetura* *Tecnologia*
Isto é na implementação adotada para cada nível

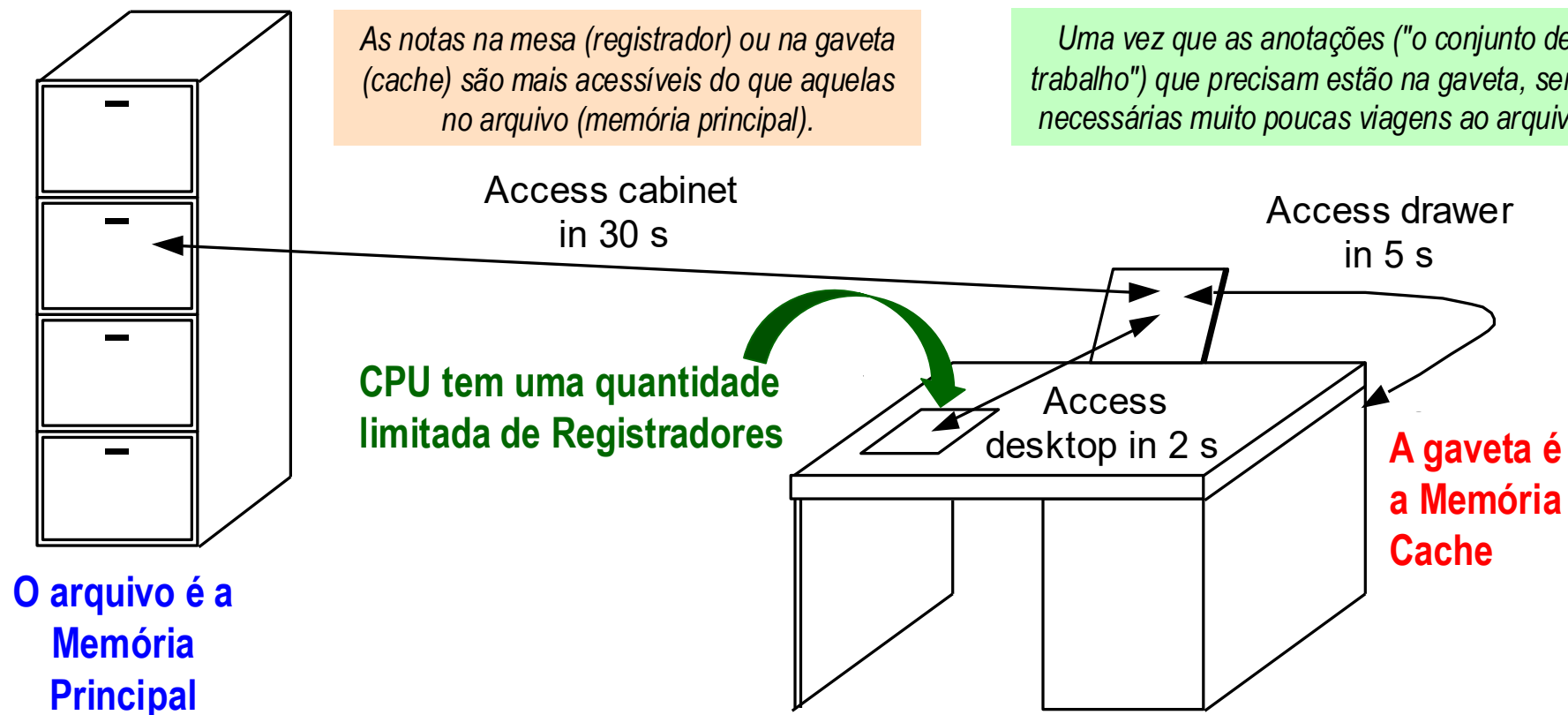
- Como melhorar o desempenho de programas Von Neumann?
 - *"Bend the rules?" = Vergar as regras?*

Conceito de Localidade



Memoria Cache - Uma analogia

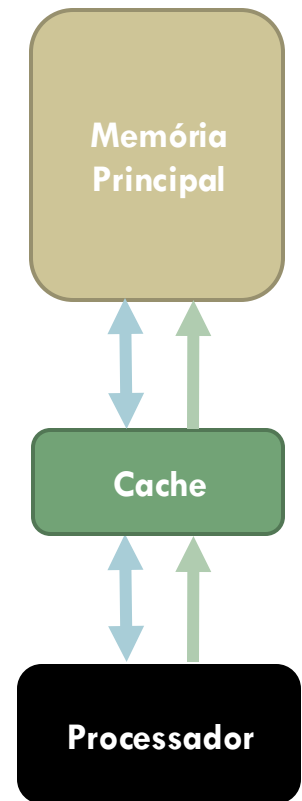
Fundamentos de Arquiteturas de Computadores



- ❑ Você está fazendo uma avaliação, onde quer colocar suas anotações:
 - Na mesa (*desktop*) que representa o CPU onde cabe uma folha por vez;
 - Na gaveta (*drawer*) que representa uma cache, e cabe algumas folhas; ou
 - No armário de arquivo (*cabinet*) que representa a memória principal?

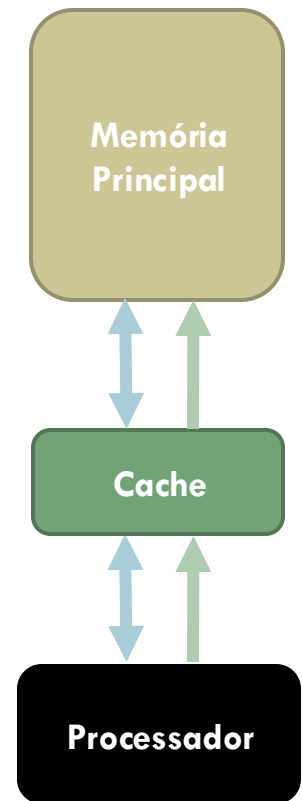
Aproveitando a **Localidade Temporal**

- Na primeira vez que o processador pede para ler de um endereço na memória principal, uma cópia desse dado também é armazenada na cache.
- Na próxima vez que o mesmo endereço for lido, podemos usar a cópia do dado na cache em vez de acessar a MP mais lenta.
- Portanto, a primeira leitura é um pouco mais lenta do que sem cache, pois passa pela memória principal e pelo cache, mas as leituras subsequentes são muito mais rápidas.
- Isso aproveita a localidade temporal - os dados comumente acessados são armazenados na memória cache para ser acessada mais rapidamente no futuro.



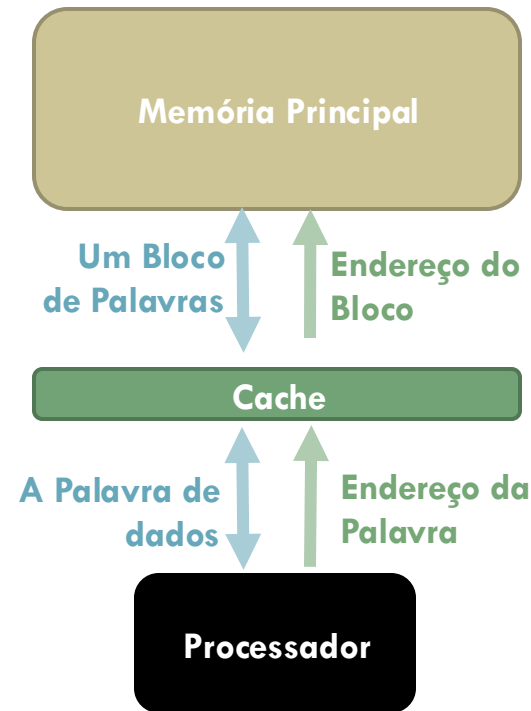
Aproveitando a Localidade Temporal

- ❑ O tempo total (TT) para busca o dado no mesmo endereço K vezes, sem o uso de um cache
 - $TT_{MP}(K) = K \times TA_{MP}$
- ❑ O tempo para a primeira busca usando um cache
 - $TT_C(1) = TA_C + TA_{MP}$
- ❑ Como a cache fica com uma cópia do dado recente buscado, requisições subsequentes vão só precisa acessar a cache:
 - $TT_C(K) = TT_C(1) + TT_C(K-1) = TA_C + TA_{MP} + (K - 1)TA_C$
- ❑ Então, qual é o benefício de usar uma cache?
 - $Speedup = TT_{MP}(K) \div TT_C(K)$
 - $= (K \times TA_{MP}) \div (TA_{MP} + K \times TA_C)$
 - $= TA_{MP} \div TA_C$ para $K \rightarrow \infty$
 - Se $TA_{MP} = 100 \times TA_C$, o *Speedup* poderia chegar quase 100



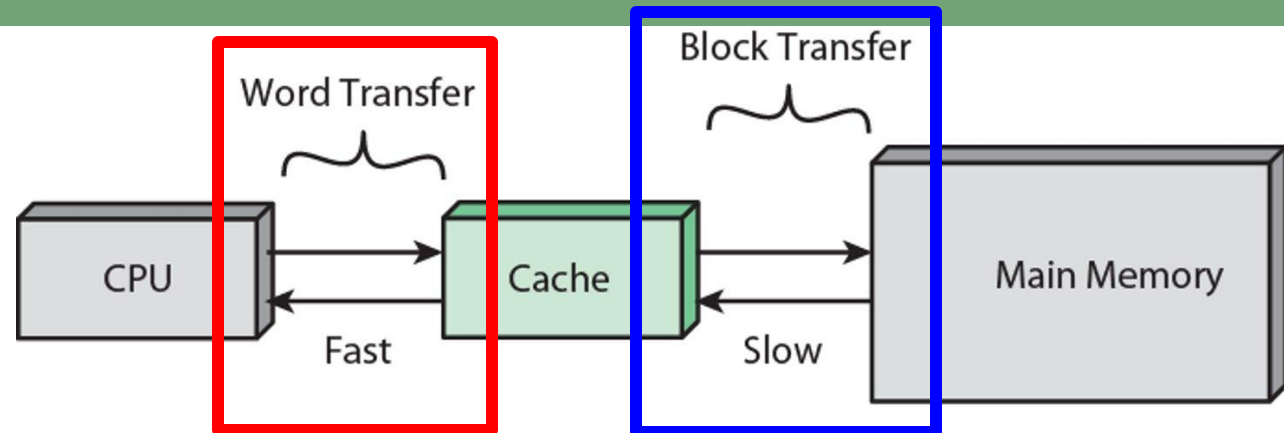
Aproveitando a Localidade Espacial

- Quando a CPU lê um endereço na memória principal, uma cópia da palavra de dados é colocada na cache.
- Mas, em vez de apenas copiar só uma palavra, as palavras de endereços vizinhos na MP também são copiadas juntas para o cache de uma vez só
- Se a CPU posteriormente precisar ler destes dados pela primeira vez, serão acessados da cache e não da MP.
 - Por exemplo, em vez de ler apenas um elemento do array por vez, o cache pode carregar, por exemplo, quatro elementos do array de uma vez.
- Novamente, a carga inicial incorre em uma penalidade de desempenho, mas estamos apostando na localidade espacial e na probabilidade de que a CPU vai precisar de um ou mais dos dados buscados.



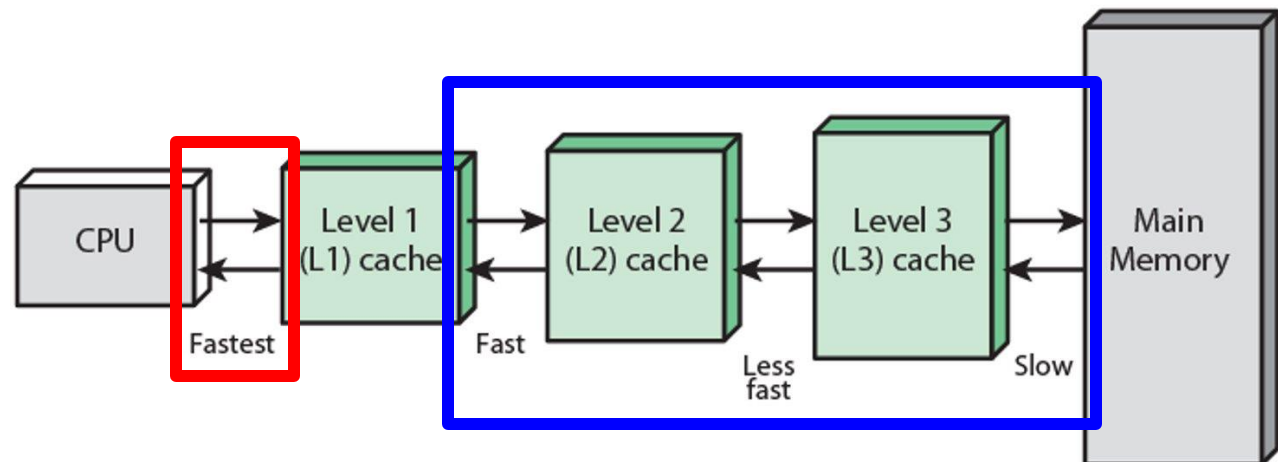
Funcionamento da Memória Cache

- **Uma palavra** de dados é transferida entre o CPU e a cache



(a) Single cache

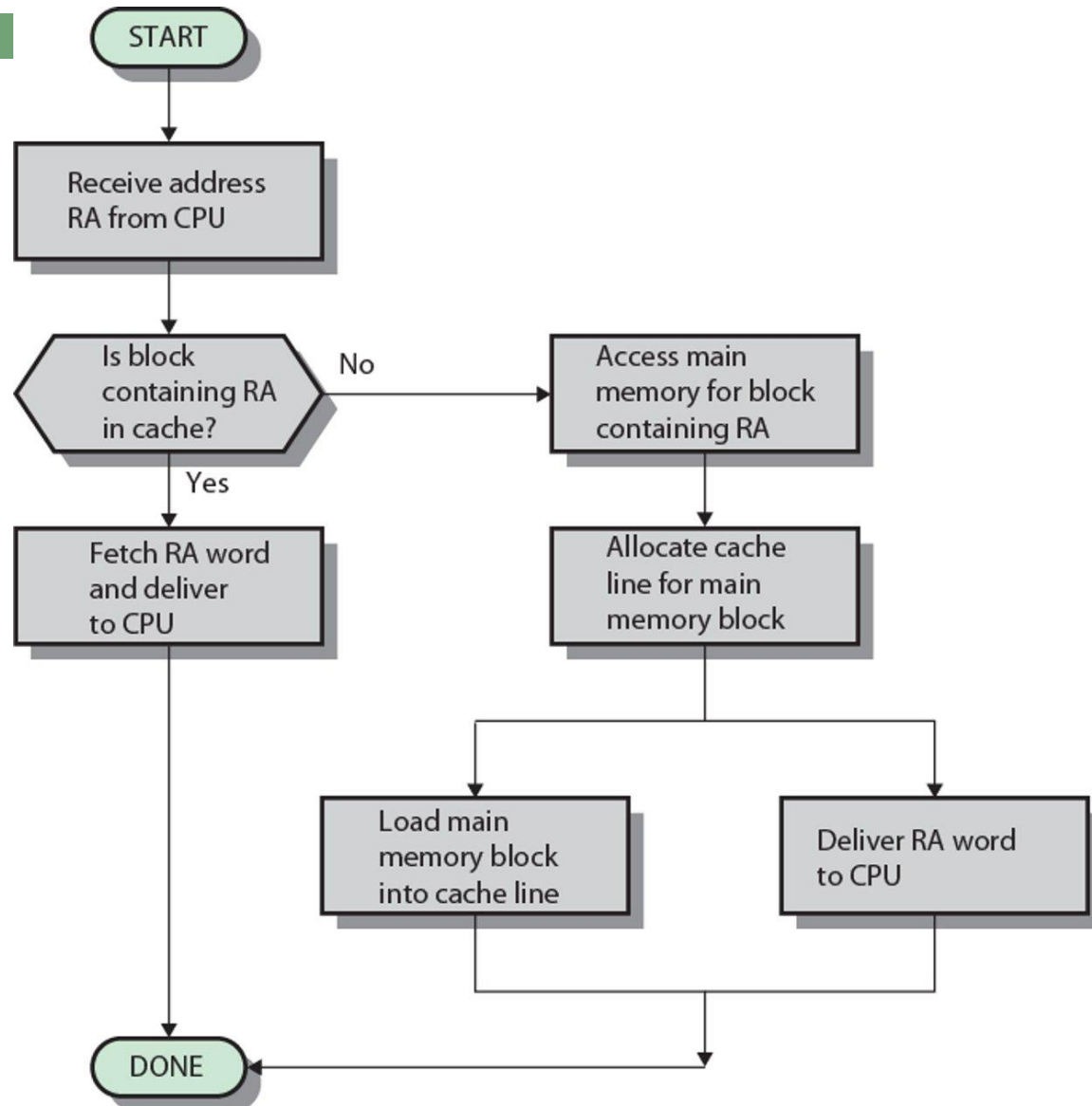
- **Um bloco de palavras** é transferido entre os níveis da cache e da memória



(b) Three-level cache organization

Funcionamento da Memória Cache

- RA = *endereço* da **palavra** solicitada
- CPU envia um endereço (RA) para o que ele acha é a memória principal
- O CPU não vai saber se o dado veio do(s) cache(s) ou a MP



Aproveitando a Localidade Espacial

Soma de Matrizes

```
for(x=0; x<n; x++)  
    for(y=0; y<n; y++)  
        c[x][y] = a[x][y] + b[x][y];
```

Aproveitando a Localidade Espacial

Soma de Matrizes

```
for(x=0; x<n; x++)  
    for(y=0; y<n; y++)  
        c[x][y] = a[x][y] + b[x][y];
```

Trocar a ordem dos dois laços **x** e **y** faria alguma diferença?

Aproveitando a Localidade Espacial

Soma de Matrizes

```
for(x=0; x<n; x++)
    for(y=0; y<n; y++)
        c[x][y] = a[x][y] + b[x][y];
```

i=	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
x,y=	0,0	1,0	2,0	3,0	0,1	1,1	2,1	3,1	0,2	1,2	2,2	3,2	0,3	1,3	2,3	3,3

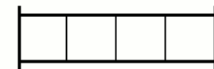
Image data laid out in one-dimensional memory

	x=0	1	2	3
y=0	0	1	2	3
1	4	5	6	7
2	8	9	10	11
3	12	13	14	15

Image data laid out in two dimensions

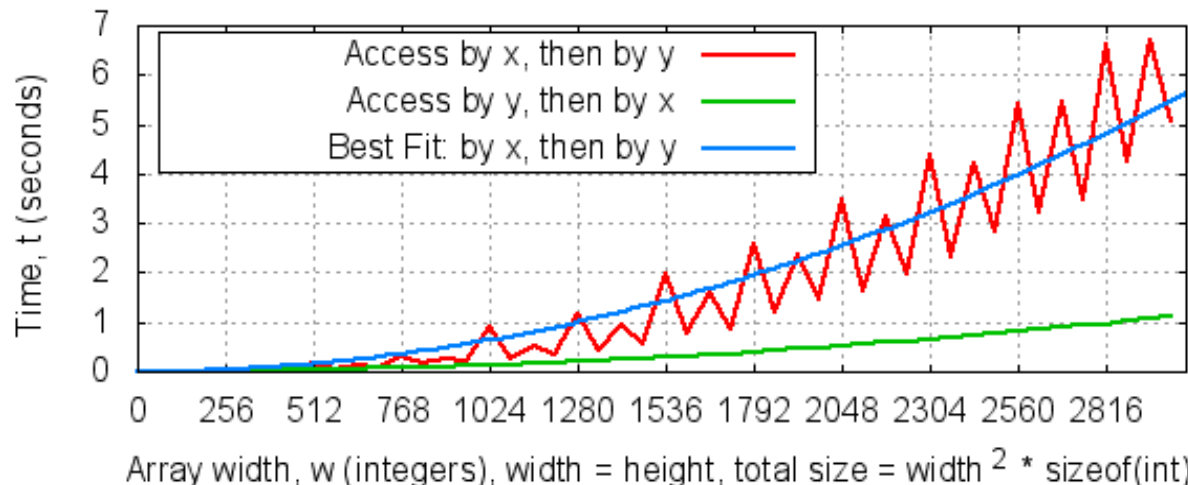


One byte of pixel data



One row of pixels

Time taken to access a two-dimensional array of size w^2 integers



Onde achamos caches?

- A ideia geral por trás das caches é usada em muitas outras situações onde têm comunicação de dados. As redes de comunicação são provavelmente o melhor exemplo.
 - As redes que têm “latência” relativamente alta e “largura de banda” baixa, portanto, transferências de dados repetidas são indesejáveis.
 - Navegadores (*Web browsers*) armazenam suas páginas da Web acessadas mais recentemente em seu disco rígido.
 - Os administradores podem configurar um cache para toda a rede internet, e empresas como a *Akamai* também fornecem serviços de cache.

Onde achamos caches?

□ Alguns outros exemplos:

- Muitos processadores têm um TLB (*Translation Lookaside Buffer*), que é um cache dedicado ao suporte de memória virtual;
- Os sistemas operacionais podem armazenar blocos de disco acessados com frequência, como diretórios, na memória principal ... e esses dados podem então ser armazenados no cache da CPU!
- A técnica de *Memoization* é usado principalmente para acelerar programas, armazenando os resultados de chamadas de funções caras e retornando o resultado previamente colocado na cache quando as mesmas entradas ocorrem novamente.

Caches: acertos e falhas

- Um acerto de cache (**cache hit**) ocorre se o cache contém o dado que estamos procurando.
 - Os acertos são bons, porque o cache pode retornar o dado muito mais rápido do que a memória principal.
- Ocorre um falha de cache (**cache miss**) se o cache não contém os dados solicitados.
 - Isso é ruim, já que a CPU deve então esperar pela memória principal mais lenta.
- Existem duas medidas básicas de desempenho do cache.
 - A taxa de acertos (**hit rate**) é a porcentagem de acessos à memória que são tratados pelo cache.
 - A taxa de falha (**miss rate = 1 – hit rate**) é a porcentagem de acessos que devem ser tratados pela memória principal
- Dado que caches tipicamente têm uma taxa de acerto de 95% ou mais, qual é o tempo efetivo de acesso à memória principal?

Caches: acertos e falhas

- Um acerto de cache (**cache hit**) ocorre se a cache contém o dado que estamos procurando.
 - Os acertos são bons, porque o cache pode retornar o dado muito mais rápido do que a memória principal.
- Ocorre um falha de cache (**cache miss**) se a cache não contém os dados solicitados.
 - Isso é ruim, já que a CPU deve então esperar pela memória principal mais lenta.
- Existem duas medidas básicas de desempenho da cache.
 - A taxa de acertos (**hit rate**) é a porcentagem de acessos à memória que são tratados pela cache.
 - A taxa de falha (**miss rate** = **1 – hit rate**) é a porcentagem de acessos que devem ser tratados pela memória principal
- *Tempo de acesso (TA) médio* = $TA_{cache} + (1 - hit\ rate) \times TA_{memória}$

Acessando Memória Principal

- O processador continua achando que ele está acessando a memória principal, enviando um endereço (com tamanho M bits) da palavra solicitada

M bits do endereço da palavra

- Porém a cache, para explorar localidade espacial, precisa acessar um conjunto de palavras – isto é um bloco.

